

Program 12:- Write a program to implement travelling salesperson problem.

```

#include <iostream>
#include <vector>
#include <limits.h>
using namespace std;
const int MAX = 1000000;
int n;
vector<vector<int>>> dist;
vector<vector<int>>> memo;
int fun(int i, int mask) {
    if (mask == ((1 << i) | 3))
        return dist[1][i];
    if (memo[i][mask] != -1)
        return memo[i][mask];
    int res = MAX;
    for (int j = 1; j <= n; j++) {
        if ((mask & (1 << j)) && j != i && j != 1)
            res = min(res, fun(j, mask & ~(1 << i)) + dist[j][i]);
    }
    return memo[i][mask] = res;
}

int main() {
    cin >> n;
    dist.resize(n + 1, vector<int>(n + 1, 0));
    memo.resize(n + 1, vector<int>(1 << (n + 1), -1));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cin >> dist[i][j];
        }
    }
    int ans = MAX;
    for (int i = 1; i <= n; i++) {
        ans = min(ans, fun(i, (1 << (n + 1)) - 1) + dist[i][1]);
    }
    cout << ans << endl;
    return 0;
}

```

Output:-

```

Enter the number of nodes: 4
Enter the distance matrix (0 for same node):
0 10 15 20
15 25 0 30
10 0 25 25
20 25 30 0
The cost of the most efficient tour = 55

```

Program 11:- Write a Program to implement N Queen Problem using Backtracking.

```
#include <bits/stdc++.h>
using namespace std;
void printSolution(vector<vector<int>>& board) {
    int n = board.size();
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            if (board[i][j])
                cout << "Q ";
        else
            cout << ". ";    cout << "\n"; }
bool isSafe(vector<vector<int>>& board, int row, int col) {
    int n = board.size();
    for (int i = 0; i < col; i++)
        if (board[row][i])
            return false;
    for (int i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return false;
    for (int i = row, j = col; j >= 0 && i < n; i++, j--)
        if (board[i][j])
            return false;
    return true;}
bool solveNQUtil(vector<vector<int>>& board, int col) {
    int n = board.size();
    if (col >= n)
        return true;
    for (int i = 0; i < n; i++) {
        if (isSafe(board, i, col)) {
            board[i][col] = 1;
            if (solveNQUtil(board, col + 1))
                return true;
            board[i][col] = 0; }
    return false;}
bool solveNQ(int n) {
    vector<vector<int>> board(n, vector<int>(n, 0));
    if (!solveNQUtil(board, 0)) {
        cout << "Solution does not exist\n";
        return false;}
    printSolution(board);
    return true;}
int main() {
    int n;
    cout << "Enter the size of the board (n): ";
    cin >> n;
    if (n <= 0) {
        cout << "Invalid board size. Please enter a positive integer.\n";
        return 1; }
    solveNQ(n);
    return 0;}
```

Output:-

```
Enter the size of the board (n): 4
. . Q .
Q . . .
. . . Q
. Q . .
```